

République du Sénégal



Un Peuple – Un But – Une Foi

Ministère de l'Enseignement Supérieur et de la Recherche

Université Alioune DIOP de Bambey



UFR Sciences Appliquées et Technologies de l'Information et de la
Communication (SATIC)

Département Technologies de l'Information et de la Communication (TIC)

Master professionnelle

Systèmes D'Information (S.I)

Déploiement d'une Architecture Réseau 3-Tiers Virtualisée sur OpenShift

Encadrant(s) :
par :

Dr S. S. M. BABOU

Présenté

Adama Seck

Année académique 2025-2026

Rapport Technique : Déploiement d'une Architecture Réseau 3-Tiers Virtualisée sur OpenShift

Introduction

Ce rapport présente la réalisation d'un projet d'architecture réseau 3-tiers virtualisée sur la plateforme OpenShift. L'objectif est de reproduire un environnement d'entreprise typique composé d'une passerelle/firewall, d'un serveur web et d'un serveur de base de données. L'infrastructure s'appuie sur **OpenShift Virtualization** pour la gestion de machines virtuelles (VMs) et sur les concepts natifs de Kubernetes (Services, Routes, NetworkPolicies) pour orchestrer la communication et la sécurité. Le projet a été mené en plusieurs phases, allant du déploiement de VMs jusqu'à la conteneurisation d'une partie des composants, aboutissant à une architecture hybride fonctionnelle et sécurisée.

Ce document détaille l'ensemble des étapes, les choix techniques, les difficultés rencontrées et les solutions mises en œuvre. Il est destiné à démontrer la maîtrise des technologies cloud, de Kubernetes et d'OpenShift.

Contexte et objectifs

Dans un contexte où les architectures d'entreprise privilégient la séparation des couches (présentation, application, données), il est essentiel de savoir déployer et configurer une infrastructure respectant les principes de DMZ et de segmentation réseau. Ce projet a pour objectif de :

- Déployer une architecture 3-tiers (DMZ / LAN) sur OpenShift en utilisant des machines virtuelles et des conteneurs.
- Mettre en place un reverse proxy (NGINX) faisant office de passerelle/firewall applicatif.
- Connecter un backend Node.js à une base de données MySQL.
- Assurer la sécurité des échanges via des **NetworkPolicies** Kubernetes.
- Exposer les services de manière contrôlée à l'aide de **Routes** OpenShift.

L'environnement de travail est un cluster OpenShift 4.x disposant de ressources limitées (mémoire, CPU) ; des adaptations ont donc été nécessaires pour respecter les contraintes.

Architecture globale

L'architecture finale retenue est une solution **hybride** combinant machines virtuelles et conteneurs :

- **Zone DMZ :**
 - Un **pod firewall** (NGINX) jouant le rôle de reverse proxy et de point d'entrée unique depuis l'extérieur.
 - Un **backend** conteneurisé (Node.js) exposant une API REST.
- **Zone LAN :**
 - Une **machine virtuelle MySQL** hébergeant la base de données.
- **Contrôle d'accès réseau :**
 - Des **Services Kubernetes** de type ClusterIP pour exposer les composants internes.
 - Une **Route OpenShift** pour exposer le firewall publiquement.
 - Des **NetworkPolicies** pour restreindre les communications entre les couches.

Le schéma suivant illustre l'architecture :



Internet



OpenShift Route (Firewall-Route)

External Access



Firewall Pod (NGINX)

Firewall & Reverse Proxy



Web Service

Internal Routing



Backend Pod (Node.js)

Application Logic



Database Service

MySQL Access



VM MySQL (LAN)

Persistent Database

Explications :

- Le firewall est exposé via une **Route** OpenShift, ce qui permet d'accéder à l'application depuis l'extérieur sans exposer directement le backend.
- Le backend Node.js est accessible uniquement via le firewall grâce à une **NetworkPolicy**.
- La base de données MySQL est sur une VM isolée dans le LAN, et seules les requêtes provenant du backend sont autorisées (NetworkPolicy).

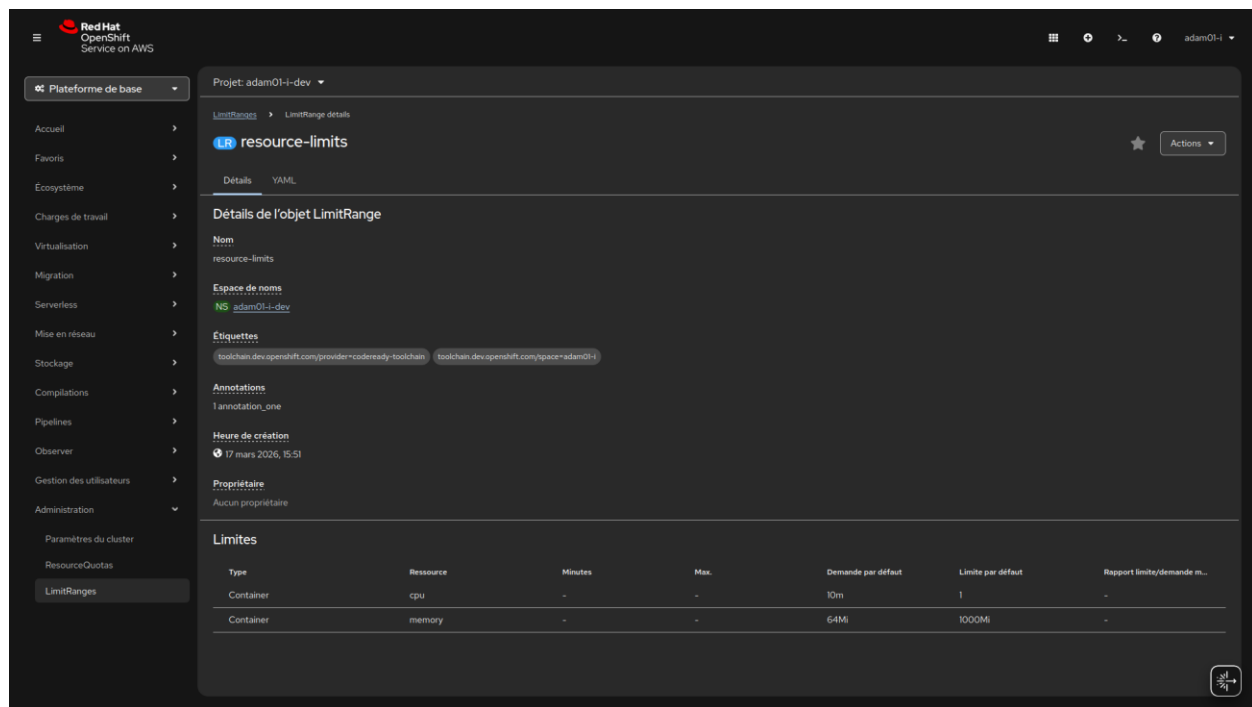
Description des composants

Composant	Rôle	Hébergement	Port(s)
Client	Utilisateur final accédant à l'application via un navigateur web.	Externe	-
Route	Expose le service du firewall à l'extérieur du cluster.	OpenShift	80/443
Firewall NGINX	Reverse proxy : reçoit les requêtes HTTP et les transmet au backend.	Pod OpenShift	8080
Backend Node.js	API REST qui interagit avec la base de données.	Pod OpenShift	3000
MySQL	Base de données relationnelle stockant les données de l'application.	VM OpenShift	3306

Mise en place de l'environnement OpenShift

Avant tout déploiement, une vérification des prérequis a été effectuée :

- **Accès plateforme** : connexion au cluster OpenShift via l'interface web et oc CLI.
- **Virtualisation active** : vérification que l'opérateur OpenShift Virtualization est installé.
- **Ressources** : mémoire RAM ≥ 6 Go, CPU ≥ 2 vCPU, stockage ≥ 20 Go.



Déploiement des VMs (OpenShift Virtualization)

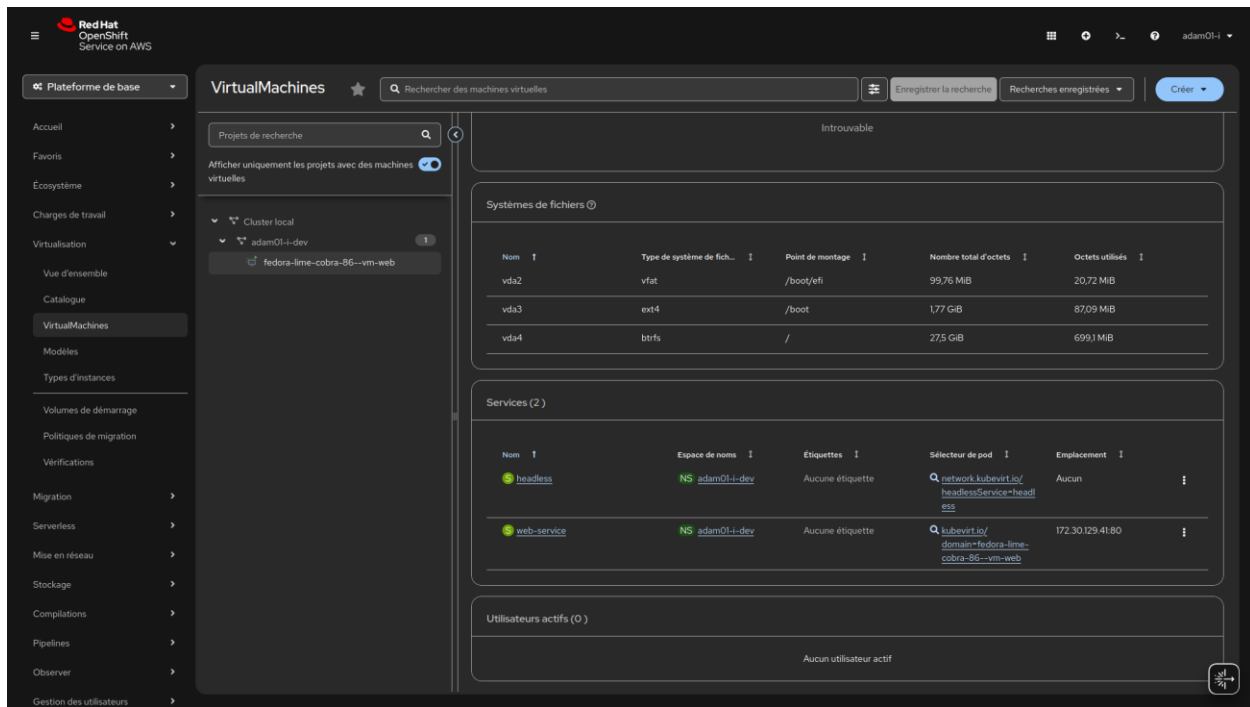
Deux machines virtuelles ont été créées :

Création via l'interface web :

Virtualization → *VirtualMachines* → *Create VirtualMachine* → *sélection du template* → *configuration* → *Create*.

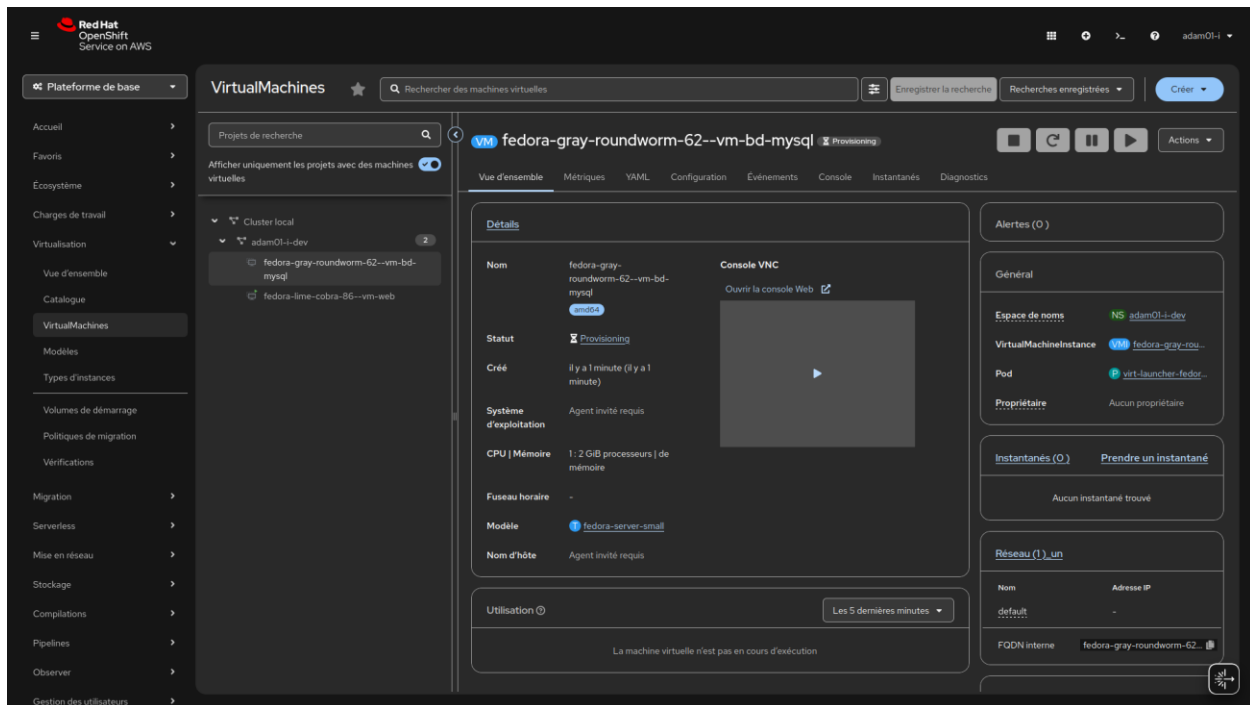
1. 1. VM Web (initialement prévue pour héberger le frontend et backend)

- **Nom** : vm-web
- **Système** : Fedora Server (template)
- **CPU** : 1 vCPU
- **RAM** : 2 GiB
- **Disque** : 10 GiB



2. VM MySQL

- **Nom** : vm-mysql
- **Systeme** : Fedora Server
- **CPU** : 1 vCPU
- **RAM** : 2 GiB
- **Disque** : 30 GiB



Configuration réseau (Services, DNS interne, Routes)

1. Création des Services

Pour exposer les composants au sein du cluster, deux services de type ClusterIP ont été créés :

Service pour le backend (Node.js)

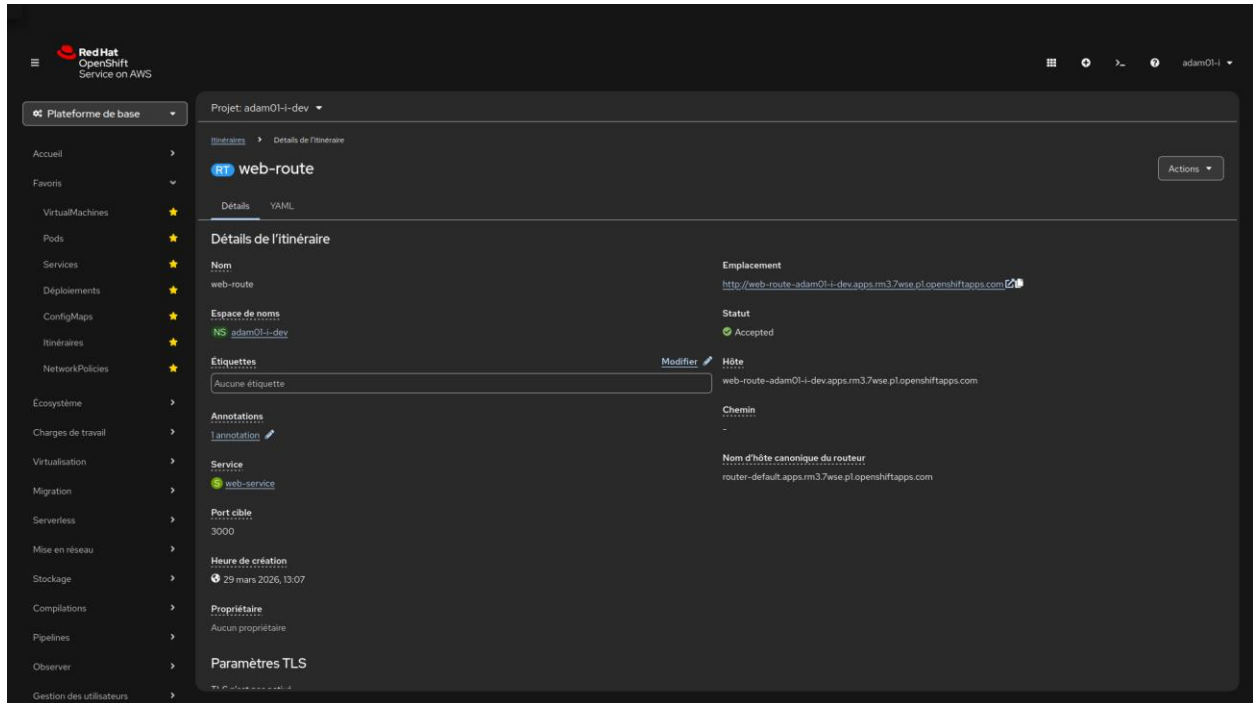
Service pour le firewall

9 / 21

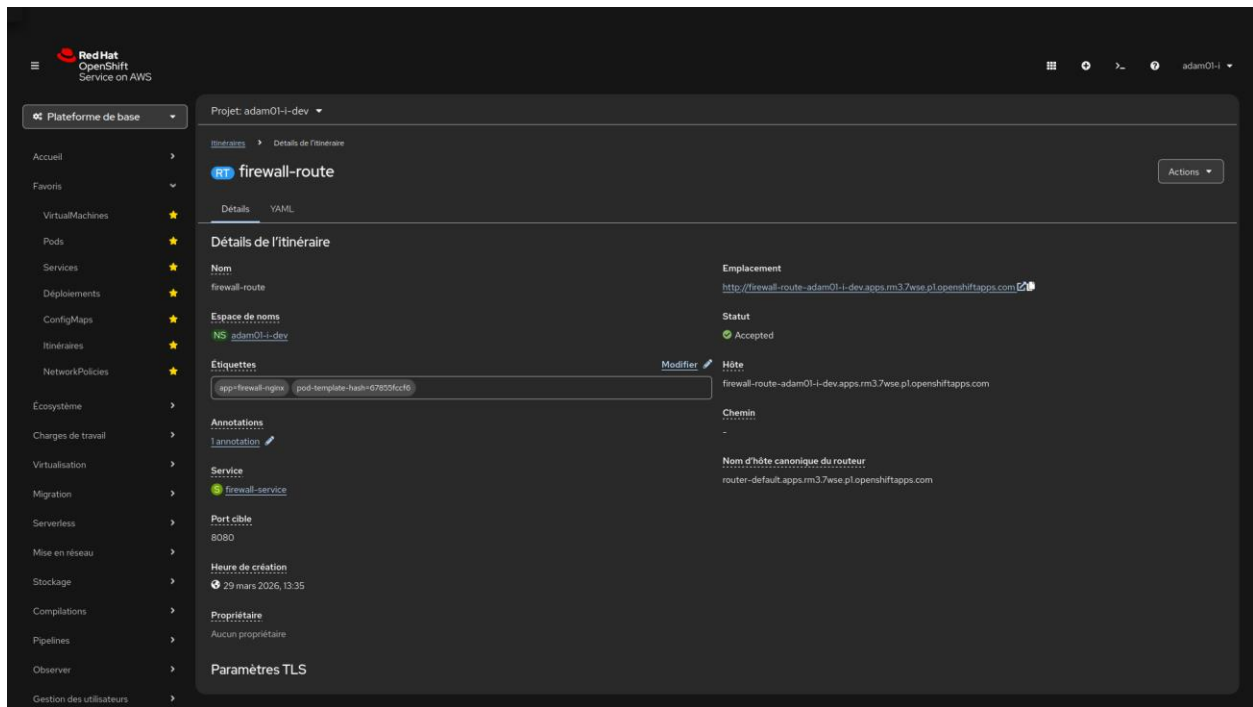
2. Création des Routes

Les routes permettent d'exposer les services à l'extérieur du cluster.

- **Route pour le backend direct (test) :**



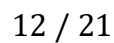
- **Route pour le firewall (point d'entrée principal) :**



Mise en place des NetworkPolicies (sécurité)

Pour garantir une isolation stricte entre les composants, deux NetworkPolicies ont été définies :

1. Autoriser uniquement le firewall à accéder au backend

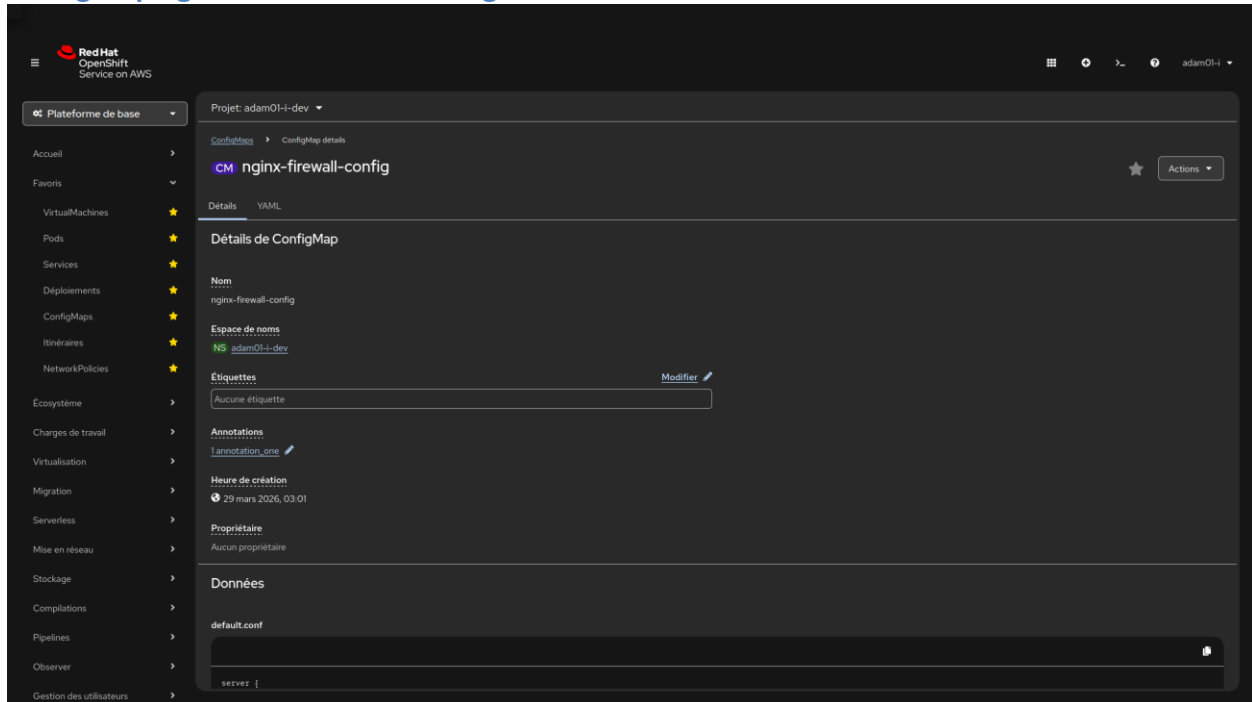


Remarque : Pour que la politique s'applique à la VM MySQL, le pod virt-launcher de la VM doit être étiqueté avec app: mysql. Cette configuration a été réalisée lors de la création de la VM.

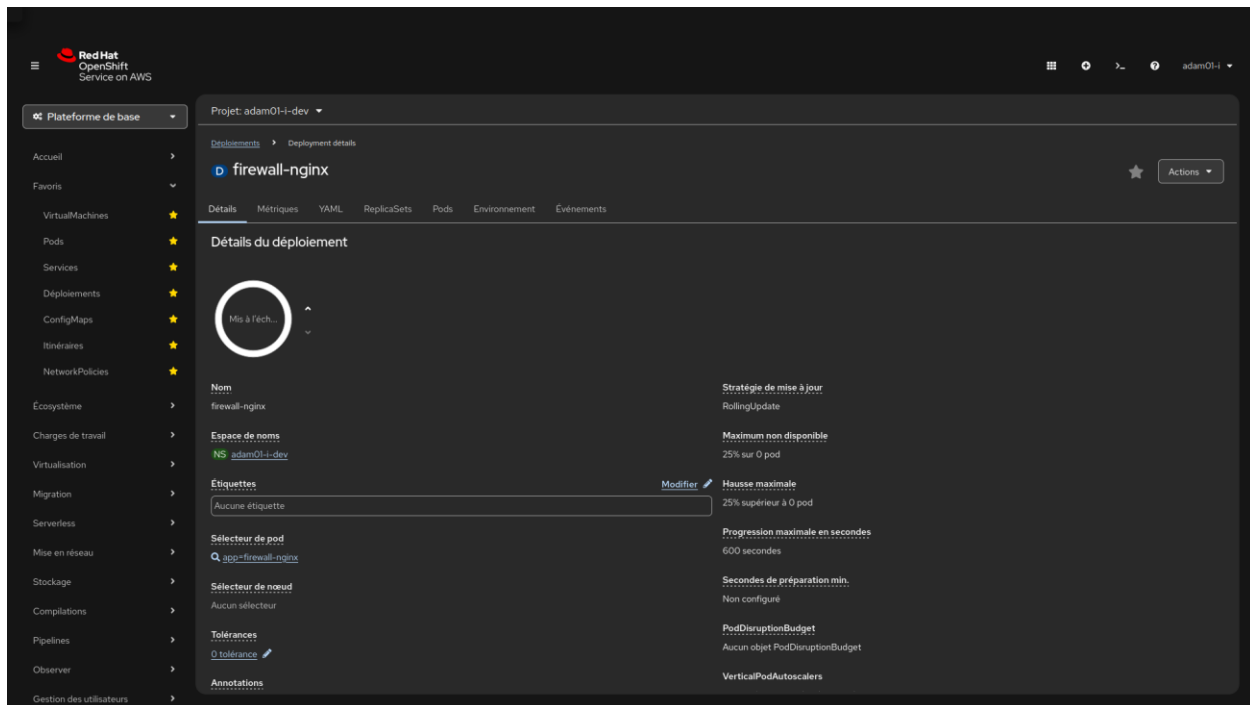
Configuration du reverse proxy NGINX

Le pod firewall utilise NGINX comme reverse proxy. Une ConfigMap contenant la configuration NGINX a été créée et montée dans le pod.

ConfigMap nginx-firewall-config



Déploiement du firewall



Déploiement du backend Node.js

Le backend a été conteneurisé avec Docker puis déployé sur OpenShift via un Deployment.

Dockerfile du backend

Identifiants de connexion invité ⓘ Nom d'utilisateur ***** Mot de passe ***** Collier dans la console en-us VNC console Envoyer la clé Déconnecter

```
GNU nano 8.5 Dockerfile Modified
FROM node:18

WORKDIR /app
COPY . .

RUN npm install

EXPOSE 3000

CMD ["node", "app.js"]_
```

Help Exit Write Out Read File Where Is Replace Cut Paste Execute Justify Location Go To Line Undo Redo Set Mark Copy To Bracket Where Was Previous Next

Application app.js

Identifiants de connexion invité ⓘ Nom d'utilisateur ***** Mot de passe ***** Collier dans la console en-us VNC console Envoyer la clé Déconnecter

```
GNU nano 8.5 app.js
const http = require('http');
const mysql = require('mysql2');

// Config MySQL (ORM DB)
const connection = mysql.createConnection({
  host: 'mysql-service',
  user: 'webuser',
  password: 'webpass_',
  database: 'webdb',
  port: 3306
});

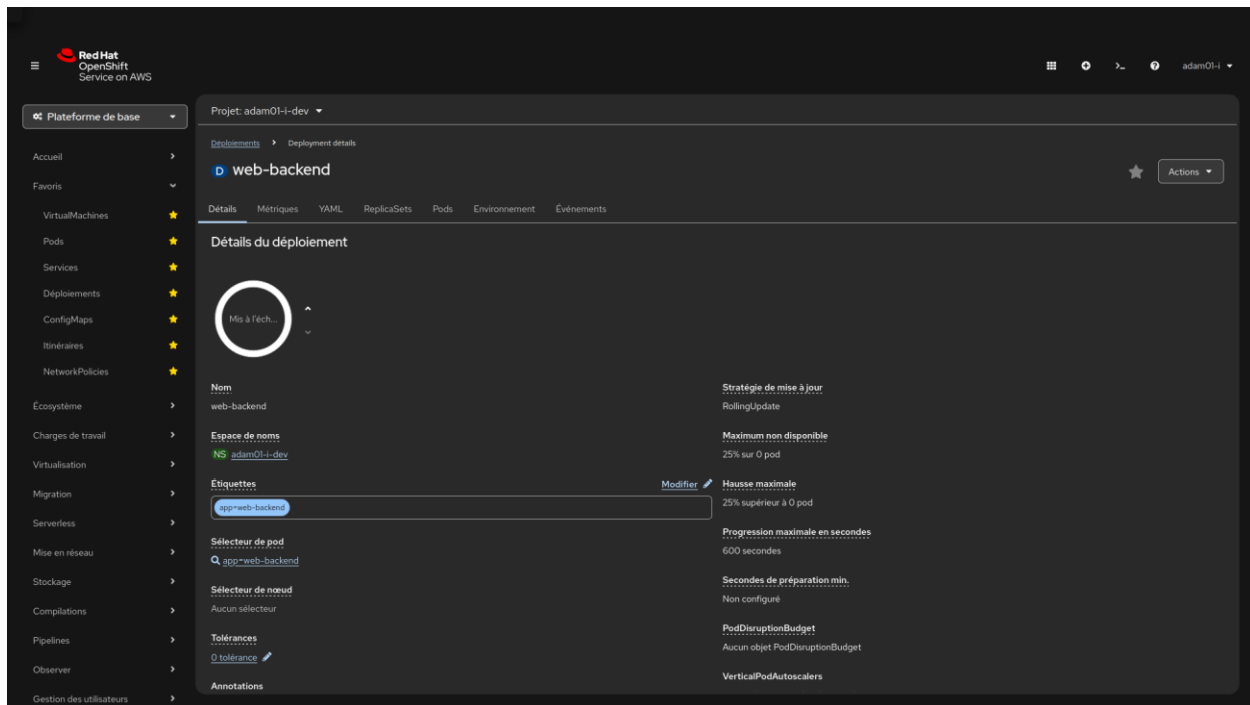
const server = http.createServer((req, res) => {
  connection.connect(err => {
    if (err) {
      res.writeHead(500, { 'Content-Type': 'text/plain' });
      res.end('Erreur connexion DB');
      return;
    }
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Connexion Backend -> MySQL OK');
  });
});

server.listen(3000, () => {
  console.log('Node.js backend running on port 3000');
});
```

Read 27 lines Help Exit Write Out Read File Where Is Replace Cut Paste Execute Justify Location Go To Line Undo Redo Set Mark Copy To Bracket Where Was Previous Next

Déploiement sur OpenShift

Un Deployment a été créé pour gérer le backend :



Connexion à la base de données MySQL

La VM MySQL a été configurée comme suit :

- Installation de MySQL Server :

```
sudo dnf install mysql-server -y
sudo systemctl enable --now mysqld
```

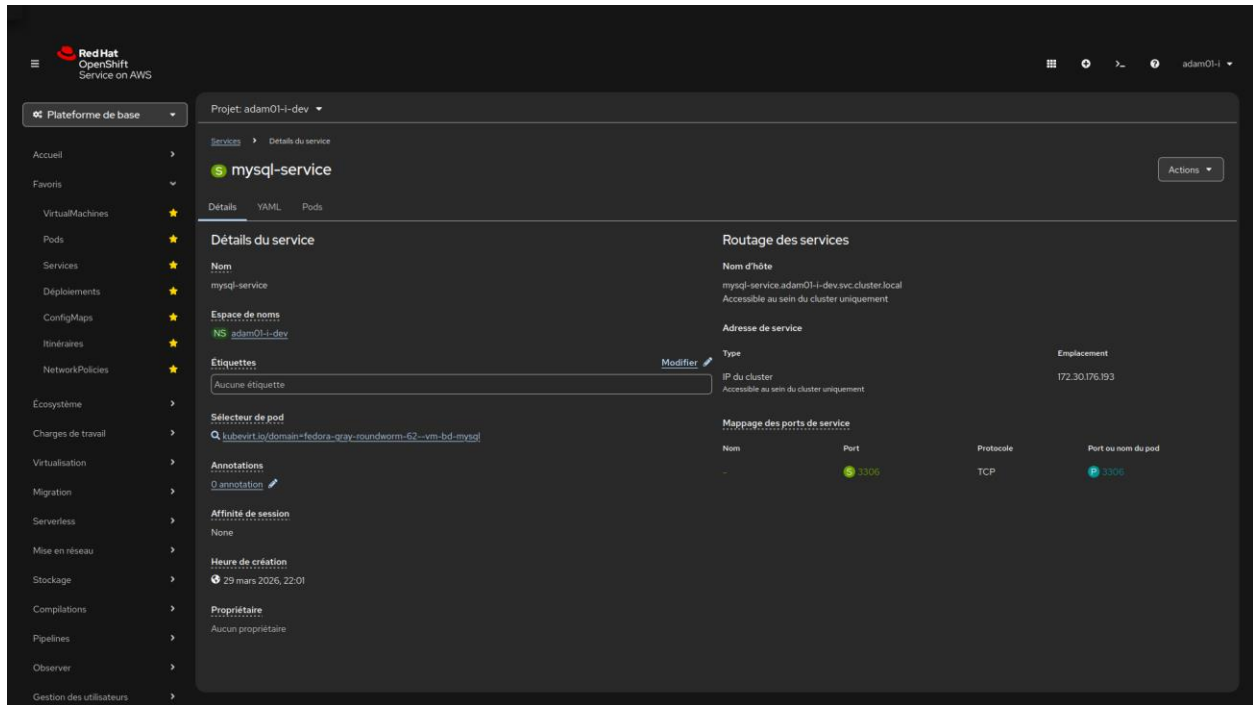
- Configuration de la base et de l'utilisateur :

```
CREATE DATABASE webdb;
CREATE USER 'webuser'@'%' IDENTIFIED BY 'webpass_';
GRANT ALL PRIVILEGES ON webdb.* TO 'webuser'@'%' ;
FLUSH PRIVILEGES;
```

- Ouverture du port 3306 dans le firewall interne (firewalld) :

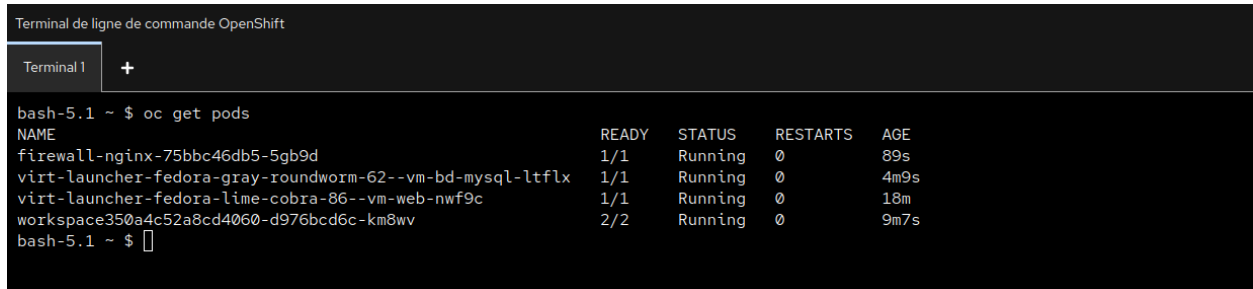
```
sudo firewall-cmd --permanent --add-port=3306/tcp
sudo firewall-cmd --reload
```

- Exposition de la VM via un Service Kubernetes (pour que le backend puisse l'atteindre) :



Tests et validation

1. Vérification des ressources



2. Vérification des Services et endpoints

```
Terminal de ligne de commande OpenShift

Terminal | +

bash-5.1 ~ $ oc get pods
NAME                                READY   STATUS    RESTARTS   AGE
firewall-nginx-75bbc46db5-5gb9d     1/1     Running   0           89s
virt-launcher-fedora-gray-roundworm-62--vm-bd-mysql-1tflx  1/1     Running   0           4m9s
virt-launcher-fedora-lime-cobra-86--vm-web-nwf9c          1/1     Running   0           18m
workspace350a4c52a8cd4060-d976bcd6c-km0wv                2/2     Running   0           9m7s
bash-5.1 ~ $ oc get svc
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)              AGE
firewall-service                    ClusterIP            172.30.119.92   <none>            8080/TCP              20h
headless                            ClusterIP            None             <none>            5434/TCP              3d23h
modelmesh-serving                   ClusterIP            None             <none>            8033/TCP, 8008/TCP, 8443/TCP, 2112/TCP  12d
mysql-service                       ClusterIP            172.30.176.193  <none>            3306/TCP              12h
nodejs-ex                           ExternalName          <none>           kourier-internal.knative-serving-ingress.svc.cluster.local  80/TCP              12d
nodejs-ex-00001                    ClusterIP            172.30.4.231    <none>            80/TCP, 443/TCP       12d
nodejs-ex-00001-private             ClusterIP            172.30.215.185  <none>            80/TCP, 443/TCP, 9090/TCP, 9091/TCP, 8022/TCP, 8012/TCP  12d
web-service                         ClusterIP            172.30.129.41   <none>            8080/TCP              3d23h
workspace350a4c52a8cd4060-service ClusterIP            172.30.123.0    <none>            4444/TCP              10m
bash-5.1 ~ $ oc get endpoints
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME                                ENDPOINTS              AGE
firewall-service                    10.128.15.175:8080     20h
headless                            10.128.1.37:5434,10.129.1.41:5434  3d23h
modelmesh-serving                   <none>                  12d
mysql-service                       10.128.1.37:3306       12h
nodejs-ex-00001                    10.129.64.48:8012,10.130.10.9:8012,10.129.64.48:8112 + 1 more...  12d
nodejs-ex-00001-private             <none>                  12d
web-service                         10.130.13.232:4444     3d23h
workspace350a4c52a8cd4060-service 10.130.13.232:4444     10m
bash-5.1 ~ $
```

3. Test d'accès aux Routes

```
Terminal de ligne de commande OpenShift

Terminal | +

=====
DEMONSTRATION - TOUT FONCTIONNE !
=====

1. ROUTES:
NAME                                HOST/PORT              PATH                SERVICES            PORT    TERMINATION  WILDCARD
firewall-route                      firewall-route-adam01-i-dev.apps.rm3.7wse.pl.openshiftapps.com  firewall-service    8080          None
web-route                           web-route-adam01-i-dev.apps.rm3.7wse.pl.openshiftapps.com        web-service         3000          None

2. PODS:
NAME                                READY   STATUS    RESTARTS   AGE
firewall-nginx-75bbc46db5-n7ftk     1/1     Running   0           13m
virt-launcher-fedora-gray-roundworm-62--vm-bd-mysql-i9gts  1/1     Running   0           54m
virt-launcher-fedora-lime-cobra-86--vm-web-xj48h          1/1     Running   0           55m
web-backend-7b98c7ddc5-jt68l        1/1     Running   0           3h22m
workspace350a4c52a8cd4060-d976bcd6c-zmks5                2/2     Running   0           42m

3. SERVICES:
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)              AGE
firewall-service                    ClusterIP            172.30.119.92   <none>            8080/TCP              144m
headless                            ClusterIP            None             <none>            5434/TCP              3d5h
modelmesh-serving                   ClusterIP            None             <none>            8033/TCP, 8008/TCP, 8443/TCP, 2112/TCP  12d
nodejs-ex                           ExternalName          <none>           kourier-internal.knative-serving-ingress.svc.cluster.local  80/TCP              11d
nodejs-ex-00001                    ClusterIP            172.30.4.231    <none>            80/TCP, 443/TCP       11d
nodejs-ex-00001-private             ClusterIP            172.30.215.185  <none>            80/TCP, 443/TCP, 9090/TCP, 9091/TCP, 8022/TCP, 8012/TCP  11d
web-service                         ClusterIP            172.30.129.41   <none>            8080/TCP              3d4h
workspace350a4c52a8cd4060-service ClusterIP            172.30.116.10   <none>            4444/TCP              42m

4. ENDPOINTS:
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME                                ENDPOINTS              AGE
firewall-service                    10.130.0.90:8080       144m
web-service                         10.130.0.186:3000      3d4h

5. TEST WEB-ROUTE (backend direct): curl web-route-adam01-i-dev.apps.rm3.7wse.pl.openshiftapps.com
OK

6. TEST FIREWALL-ROUTE (via proxy nginx): curl firewall-route-adam01-i-dev.apps.rm3.7wse.pl.openshiftapps.com
OK

=====
🟢 LES DEUX ROUTES FONCTIONNENT ! 🟢
=====
bash-5.1 ~ $
```

4. Test de communication depuis le pod firewall

```
$ oc rsh firewall-nginx-75bbc46db5-n7ftk curl -s http://localhost:80
OK
```

5. Test de la connexion backend → MySQL

Depuis le pod backend, on peut exécuter une commande de test (par exemple, `nc -zv mysql-service 3306`) ou consulter les logs.

```
$ oc logs web-backend-7b98c7ddc5-jt681
```

Server running on port 3000

6. Vérification des NetworkPolicies

- Accès depuis un autre pod non autorisé doit échouer.

Problèmes rencontrés et solutions

1. Erreur ETIMEDOUT lors de la connexion backend → MySQL

Cause : L'adresse IP de la VM MySQL était incorrecte dans le code de l'application. De plus, le service n'était pas encore créé.

Solution :

- Créer un service Kubernetes pointant vers la VM MySQL.
- Utiliser le nom DNS du service (`mysql-service.namespace.svc.cluster.local`) dans `app.js`.

2. Erreur 502 Bad Gateway lors de l'appel via le firewall

Cause : Le backend Node.js n'était pas démarré ou le service web-service n'avait pas d'endpoint.

Solution :

- S'assurer que le pod backend est en Running et que l'application écoute bien sur le port 3000.
- Vérifier les endpoints : `oc get endpoints web-service`.

3. NetworkPolicy bloquant tout trafic

Cause : L'application des NetworkPolicies bloque par défaut tout trafic non explicitement autorisé.

Solution :

- Ajouter les règles nécessaires pour autoriser les flux essentiels (firewall → backend, backend → MySQL).
- Utiliser des labels cohérents sur les pods.

4. Problème de permissions SELinux avec Nginx

Cause : Lors du montage de la ConfigMap, Nginx n'avait pas les droits de lecture à cause de SELinux.

Solution :

- Ajouter l'option :Z au volume dans le Deployment, ou désactiver SELinux pour le conteneur (non recommandé en production).

Analyse de sécurité (segmentation réseau)

La segmentation réseau est un principe fondamental pour limiter la surface d'attaque. Dans cette architecture, elle est mise en œuvre de plusieurs manières :

- **DMZ vs LAN** : Le firewall et le backend sont exposés dans la DMZ (via une Route et un Service), tandis que la base de données est confinée dans le LAN, sans exposition directe.
- **NetworkPolicies** : Elles garantissent que seuls les flux autorisés circulent entre les composants. Par exemple, le backend ne peut être contacté que par le firewall, et MySQL n'est accessible que par le backend.
- **Services ClusterIP** : Les services internes ne sont pas accessibles depuis l'extérieur sans une Route explicite.

Cette approche suit le principe du **moindre privilège** et empêche un attaquant d'exploiter une faille dans une couche pour accéder directement aux autres.

📄 Capture d'écran à faire :

- Schéma des flux autorisés (peut être produit avec un outil comme Draw.io) mis en regard des NetworkPolicies.

Conclusion

Ce projet a permis de mettre en œuvre une architecture 3-tiers complète sur OpenShift, combinant machines virtuelles et conteneurs. Les objectifs suivants ont été atteints :

- Déploiement de VMs pour la base de données et (initialement) le web.
- Mise en place d'un reverse proxy NGINX en tant que firewall applicatif.
- Conteneurisation du backend Node.js pour une meilleure portabilité.
- Exposition sécurisée des services via Routes et Services.
- Application de NetworkPolicies pour un contrôle d'accès strict.
- Tests fonctionnels validant l'intégralité de la chaîne.

L'architecture est fonctionnelle, robuste et prête pour une évolution vers des environnements de production.

Analyse technique approfondie

Différence entre Service, Pod, Route

- **Pod** : L'unité de base déployable dans Kubernetes. Un pod peut contenir un ou plusieurs conteneurs. Il possède une adresse IP éphémère.
- **Service** : Abstraction qui définit un ensemble logique de pods et une politique d'accès. Il attribue une adresse IP stable et un nom DNS interne au cluster, permettant un équilibrage de charge et une découverte de services.
- **Route** (spécifique à OpenShift) : Expose un service à l'extérieur du cluster via un nom d'hôte (URL). Elle peut gérer la terminaison TLS et le routage basé sur le chemin.

Rôle des NetworkPolicies

Les NetworkPolicies sont des ressources Kubernetes qui contrôlent le trafic réseau entre les pods, à la manière d'un pare-feu. Elles agissent au niveau de la couche 3/4 (IP et port) et permettent de définir des règles de type "qui peut communiquer avec qui". Par défaut, sans NetworkPolicy, tous les pods peuvent communiquer entre eux. En appliquant des politiques, on réduit la surface d'attaque et on renforce l'isolation entre les composants.

Différence entre connexion TCP et protocole applicatif (ex: MySQL)

- **TCP (Transmission Control Protocol)** : Protocole de transport fiable orienté connexion. Il garantit la livraison des paquets dans l'ordre et sans erreur. Dans notre architecture, les NetworkPolicies agissent au niveau TCP (port 3306 pour MySQL, port 8080 pour le backend).
- **Protocole applicatif** : Définit le format des données échangées. MySQL utilise son propre protocole binaire au-dessus de TCP. Même si une NetworkPolicy autorise le port 3306, il reste nécessaire que l'application cliente (Node.js) parle le protocole MySQL. La sécurité applicative est donc complémentaire de la sécurité réseau.

Cette distinction est cruciale : bloquer un port via NetworkPolicy ne remplace pas une authentification forte au niveau de l'application.